Sorting Algorithm Benchmark Report

Introduction

This report compares the performance of four sorting algorithms on arrays of integers using JMH. The algorithms tested are Bubble Sort, Quick Sort (with median-of-three pivot), LSD Radix Sort (base-256, 4 passes), and Java's built-in Arrays.sort().

Each algorithm was tested against four input distributions: uniform random, already sorted, reverse sorted, and nearly sorted (~1% random swaps).

Setup

- JDK: 21.0.10, Java HotSpot 64-Bit Server VM

- JMH version: 1.37

- Warmup: 3 iterations, 1 second each

- Measurement: 5 iterations, 1 second each

- Forks: 1

- Mode: Average time per operation (ms/op)

Results

Main Benchmark — 1,000,000 elements

Algorithm	Random (ms)	Sorted (ms)	Reverse (ms)	Nearly Sorted (ms)
Arrays.sort	111.23	0.98	1.80	34.29
Quick Sort	128.82	21.88	52.55	66.83
Radix Sort	9.65	37.39	37.37	39.65

Bubble Sort Benchmark — 100,000 elements

Algorithm	Random (ms)	Sorted (ms)	Reverse (ms)	Nearly Sorted (ms)
Bubble Sort	19,691.05	0.08	4,885.34	4,298.96

Bubble Sort was run separately on 100,000 elements instead of 1,000,000. At 100K elements it already takes ~20 seconds per operation on random input. Extrapolating quadratically, 1,000,000 elements would take roughly 2,000,000 ms (~33 minutes) per single invocation, making it completely impractical to benchmark at that size. Based on this evidence, Bubble Sort was excluded from the full 1M dataset.

Analysis

Radix Sort was the fastest algorithm on random data at 9.65 ms - over 10x faster than Arrays.sort. This is expected since Radix Sort runs in O(4n) time regardless of input distribution. Its performance stays nearly constant across all distributions (~37-40 ms), which confirms that it does the same amount of work no matter the input order. The reason it's slightly slower on sorted/reverse/nearly-sorted compared to random is likely due to cache access patterns - sequential integers spread more evenly across all 256 buckets, leading to more scattered memory writes.

Arrays.sort (Dual-Pivot Quicksort internally) was the clear winner on sorted and reverse-sorted input, finishing in under 2 ms. It has special optimizations to detect runs of already-ordered elements, which explains this. On random data it was competitive with Quick Sort at 111 ms. On nearly sorted data it was significantly faster than Quick Sort (34 ms vs 67 ms), again benefiting from its run-detection logic.

Quick Sort (median-of-three) performed reasonably on all distributions but was never the fastest. On sorted input it achieved 21.88 ms - much better than naive quicksort would (which would degrade to O(n^2)), confirming that median-of-three pivot selection works. However, it was still 20x slower than Arrays.sort on sorted input because it lacks the run-detection optimizations.

Bubble Sort demonstrated why O(n^2) algorithms are impractical at scale. On sorted input the early-exit optimization kicks in and it finishes in 0.08 ms (a single pass through the array). But on random input it takes nearly 20 seconds for just 100K elements. The nearly-sorted results had high variance (error of +/-9,410 ms), likely because the random swaps sometimes created long unsorted tails that took many passes to fix.

Correctness

All four algorithms were verified against Arrays.sort() as a reference. For each distribution, a copy of the array was sorted by the algorithm under test and then compared to the Arrays.sort() result using Arrays.equals(). All tests passed.

Conclusion

No single algorithm is best in all cases. Radix Sort is the fastest for large random datasets due to its O(n) complexity, but it uses extra memory and doesn't benefit from pre-sorted input. Arrays.sort is the best general-purpose choice - it's highly optimized for common patterns like sorted and nearly-sorted data while still performing well on random input. Quick Sort is a solid middle ground but can't match the JDK's heavily tuned implementation. Bubble Sort is only viable for very small or already-sorted arrays.